

Lecture

Developing Android Apps

Fragments in Android – Building Modular Uis
Dynamic and Static Fragments, Lifecycle, Communication and Use Cases

Dr. Murad Mustafa Badarna

מהו Fragment?

- A Fragment is a self-contained UI component that exists within an Activity.
- Provides a modular approach to building UIs.
- Maintains its own lifecycle, layout, and logic.
- Can be dynamically added or removed at runtime.

- **Fragment** הוא רכיב ממשק משתמש עצמאי (UI) שמנוהל בתוך Activity מאפשר לפתח ממשקים בצורה **מודולרית וגמישה**.
- ל-Fragment יש **מחזור חיים עצמאי**, **ממשק משלו** ויכולת להגיב לאירועים.
- ניתן להוסיף, להסיר או להחליף Fragments **בזמן ריצה** של האפליקציה.

מתי נשתמש ב-Fragment?

- באפליקציות עם תצוגה מרובת-פאנלים (למשל בטאבלטים).
- כשרוצים לבנות רכיבי UI לשימוש חוזר.
- כאשר יש צורך להחליף חלק ממסך מבלי לעבור ל-Activity חדש

שימוש דינמי ב- Fragment

- באפליקציות רבות אנו מעוניינים להחליף חלקים בממשק בזמן ריצה, לפי פעולת המשתמש – למשל לחיצה על כפתור.
- לצורך כך, נגדיר מיכל FrameLayout ב- XML שישמש מקום להציג Fragments
- בזמן ריצה נטען לתוכו את ה- Fragment הנבחר בעזרת FragmentManager

```
<FrameLayout ... >
    <ImageView
        android:src="@drawable/jellybean"
    ... />
</FrameLayout>
```

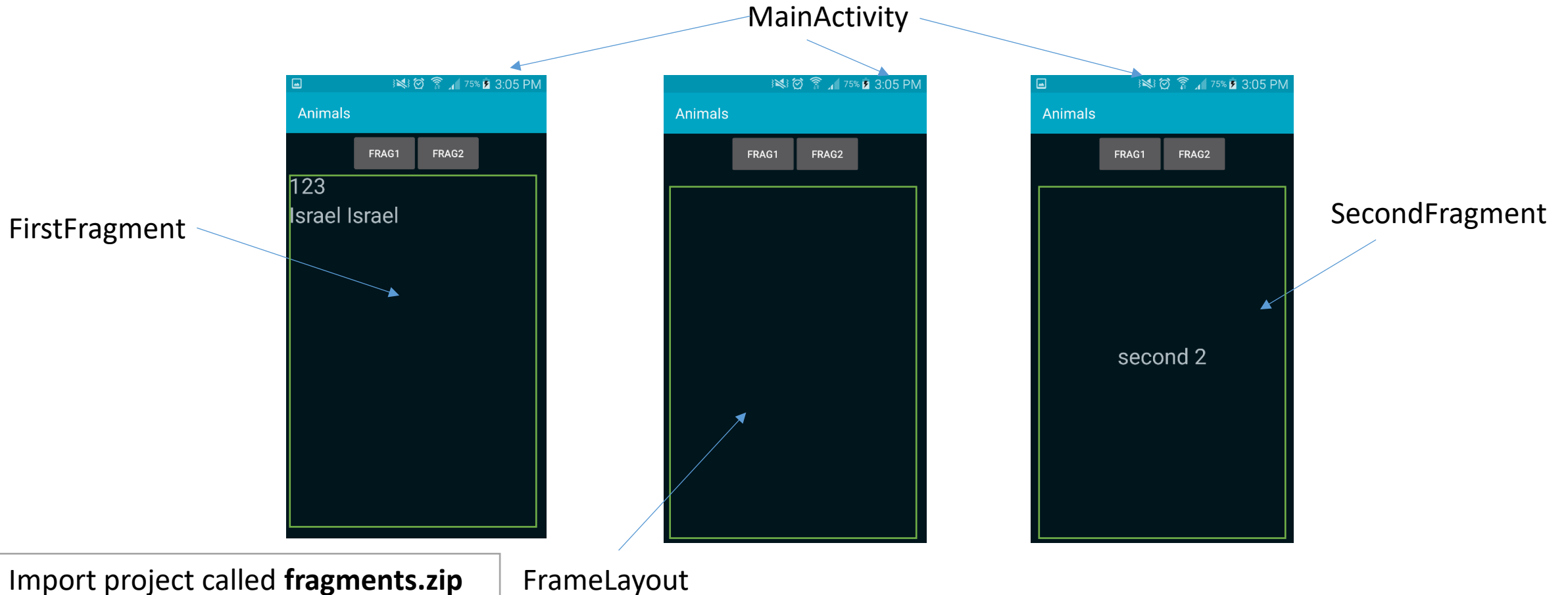


אפשר להשתמש ב FragmentContainerView במקום
FRAMELAYOUT

טעינה דינמית של Fragments בזמן ריצה באמצעות FrameLayout

טעינה דינמית של Fragments בזמן ריצה

שימוש ב- FrameLayout כמיכל והחלפת תוכן באמצעות לחצנים ו-FragmentManager



Import project called **fragments.zip**
(download from our course website)
into Android Studio

Dynamic Fragment

- Declare a `FrameLayout` place holder in the Activity's layout file

`activity_main.xml` of `MainActivity`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="top|center"
    android:orientation="horizontal">

    <Button
        android:id="@+id/btn1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/btn1_txt" />

    <Button
        android:id="@+id/btn2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/btn2_txt" />

</LinearLayout>
<FrameLayout
    android:id="@+id/root_layout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_weight="3"></FrameLayout>
```

הקוד מציג מבנה של קובץ XML בשם `activity_main.xml`, שמייצג את הממשק הגרפי של Activity אחת. הקוד כולל שני מרכיבים עיקריים:

1. `LinearLayout` אופקי עם שני כפתורים
2. בתוך ה-`LinearLayout` מופיעים שני כפתורים:
3. הם משמשים בדרך כלל להפעלת פעולות – למשל, טעינת `Fragment` שונים בלחיצה.

שימוש ב-`Fragment` – ע"י הגדרת `FrameLayout` כ-`placeholder` בתוך קובץ ה-`layout` של ה-`Activity`
זוהי אבן יסוד בממשק מבוסס `Fragment` באפליקציות מודרניות של `Android`

Dynamic Fragment

הקוד מצג כיצד ניתן להוסיף או להחליף Fragment באופן דינמי בתוך ממשק המשתמש של Activity תוך כדי ריצת האפליקציה. זהו שלב ההמשך לאחר שהוגדר מיכל FrameLayout בקובץ ה-XML

- While an Activity's running you can add a Fragment to it's layout

```
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
  
    public void showFragment2OnClick(View view) {  
        FragmentManager fm = getSupportFragmentManager();  
        SecondFragment fragmet2= new SecondFragment();  
        FragmentTransaction t= fm.beginTransaction();  
        t.replace(R.id.root_layout, fragmet2);  
        t.addToBackStack(null);  
        t.commit();  
    }  
}
```

מביא את מנגנון ניהול ה-fragments של האפליקציה.

יוצר מופע חדש של ה- Fragment הרצוי.

מתחיל שינוי שקשור ל- Fragment

מחליף את תוכן FrameLayout ב- Fragment החדש

שומר את הפעולה כך שניתן לחזור אחורה כמו בלחצן Back

מבצע בפועל את השינוי בממשק.

Dynamic Fragment

המחלקה SecondFragment יורשת מ־Fragment
הפונקציה onCreateView() היא זו שמבצעת את טעינת ממשק
המשתמש של ה־Fragment על ידי inflater

- While an Activity's running you can add a Fragment to it's layout

```
public class SecondFragment extends Fragment {  
    @Override  
    public View onCreateView(LayoutInflater inflater, ViewGroup container,  
                             Bundle savedInstanceState) {  
        // Inflate the layout for this fragment  
        return inflater.inflate(R.layout.second_fragment_layout, container, false);  
    }  
}
```

inflater הוא אובייקט מסוג
LayoutInflater, שתפקידו הוא
להמיר קובץ XML (layout) לקוד
Java שמייצר עצם של View

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    tools:context="com.haifa.animals.SecondFragment">  
  
    <!-- TODO: Update blank fragment layout -->  
    <TextView  
        android:id="@+id/mydata"  
        android:gravity="center"  
        android:textSize="30sp"  
        android:layout_width="match_parent"  
        android:text="second 2"  
        android:layout_height="match_parent"/>  
  
</LinearLayout>
```

זהו קובץ ה־UI של ה־Fragment. הוא
מגדיר ממשק פשוט. ה־TextView מיושר
למרכז, טקסט גדול, תופס את כל הרוחב
והגובה.

Send data in Bundle to fragment

```
public class MyActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_my);
    }

    public void showFragment1OnClick(View view) {
        // send data to fragment
        FragmentManager fm = getSupportFragmentManager();
        Bundle args = new Bundle();
        args.putInt("no", 123);
        args.putString("name", "Israel Israel");
        FirstFragment fragmet1= new FirstFragment();
        fragmet1.setArguments(args);
        FragmentTransaction t= fm.beginTransaction();
        t.replace(R.id.root_layout, fragmet1);
        t.addToBackStack(null);
        t.commit();
    }
}
```

כאשר נרצה ש- Fragment יקבל נתונים (כמו מזהה, שם משתמש, או כל ערך אחר) מה- Activity שמטעינה אותו – נשתמש ב- Bundle ונצמיד אותו ל- Fragment לפני ביצוע `commit()`

Get data from Bundle in Fragment

```
public class FirstFragment extends Fragment {

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        View view = inflater.inflate(R.layout.first_fragment_layout, container, false);
        TextView noTextView = (TextView) view.findViewById(R.id.noTxtView);
        TextView nameTextView = (TextView) view.findViewById(R.id.nameTxtView);

        Bundle args = getArguments();
        if(args!=null) {
            String no = String.valueOf(args.getInt("no"));
            noTextView.setText(no);
            String name = args.getString("name");
            nameTextView.setText(name);
        }
        return view;
    }
}
```

הקוד מדגים כיצד Fragment מקבל נתונים שהועברו אליו מ־Activity ומציג אותם על המסך
השיטה המרכזית: שימוש ב־Bundle + getArguments()

Add static fragment to Activity

- Declare Fragment in the Activity's layout file

main.xml of MyActivity.java

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:baselineAligned="false"
    android:orientation="horizontal" >
    <fragment
        android:id="@+id/my_fragment"
        android:layout_width="0px"
        android:layout_height="match_parent"
        android:layout_weight="1"
        class="course.examples.fragments.staticLayout.MyFragment" />
</LinearLayout>
```

אפשר להשתמש ב FragmentContainerView במקום fragment

Fragment Transactions

- ניתן להוסיף / להסיר / להחליף Fragments בזמן שה- Activity פועלת.
- כל קבוצת שינויים שמתבצעת על Activity נקראת **Transaction**
- ניתן לשמור Transaction כדי לאפשר למשתמש לנווט אחורה בין Fragments כאשר הוא לוחץ על כפתור "Back"
- אם לא נקרא **addToBackStack()** ה- Fragment ייהרס ולא ניתן יהיה לחזור אליו
- אם נקרא **addToBackStack()** ה- Fragment ייעצר ולא ייהרס, וניתן יהיה לחזור אליו כששתמש לוחץ "Back"

```
FragmentManager fm = getFragmentManager();  
FragmentTransaction t= fm.beginTransaction();  
// An optional name for this back stack state, or null  
t.addToBackStack(null);
```

Fragment Lifecycle

Containing activity has completed onCreate() and the fragment has been installed

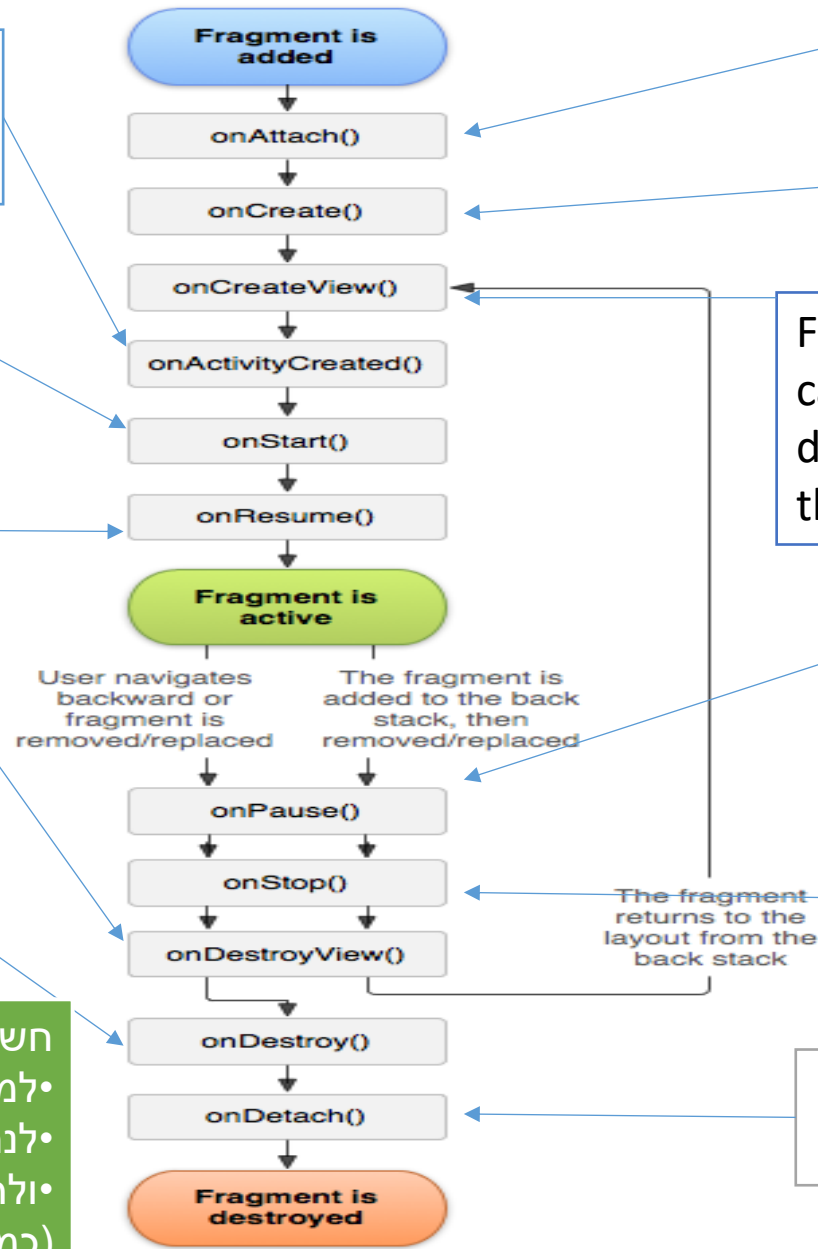
Hosting Activity about to become visible

Hosting Activity is about to become visible and ready for user interaction

View previously created by onCreateView() has been detached from the Activity.
Clean up view and resources

Fragment is no longer in use
Clean up Fragment resources

חשוב להבין את השלבים כדי:
• למנוע דליפות זיכרון,
• לנהל נתונים וממשק בצורה בטוחה,
• ולהתאים את ההתנהגות למצבי חיים שונים
(כמו שינוי סביבה, ניווט וכו').



Fragment is first attached to its activity

Initialize the Fragment. called when creating the Fragment.

Fragment sets up & returns its user Interface. called when it is time for the Fragment to draw the user interface the first time. method that must return fragment's root UI view

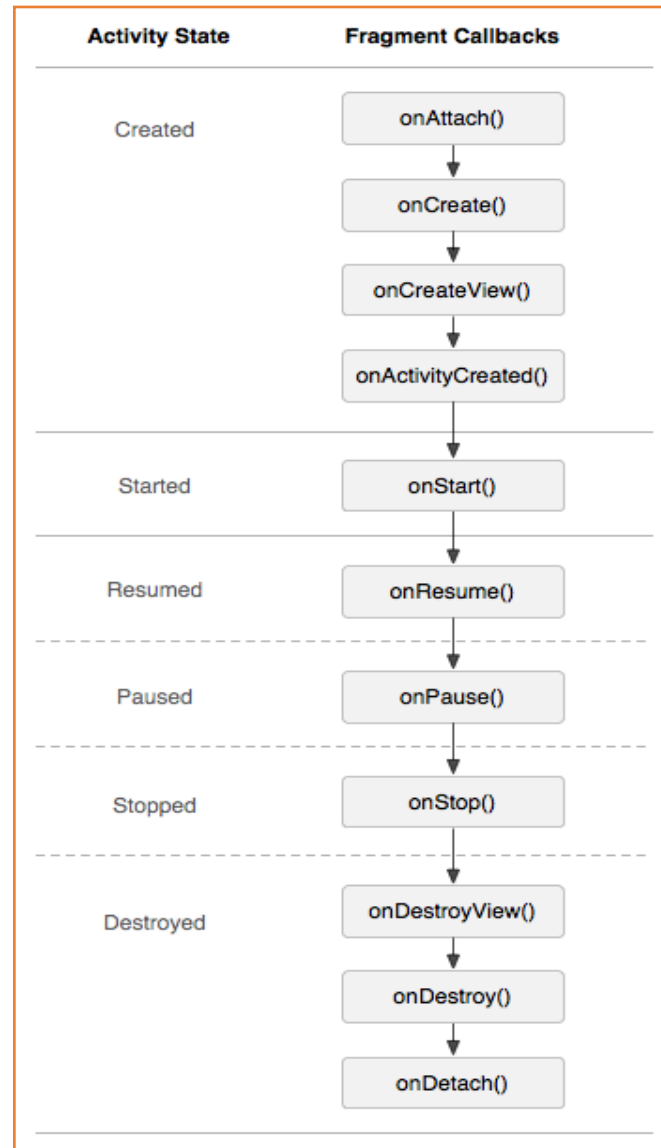
Hosting Activity is visible, but does not have focus. called when the user is leaving the Fragment.

Hosting Activity is no longer visible

Fragment no longer attached to its Activity. Null out references to Hosting Activity

Fragment Lifecycle

כל מעבר של ה-Activity בין מצבים כמו התחלה, עצירה, הרצה או הריסה – גורר אחריו קריאות מקבילות במחזור החיים של ה-**Fragment**



The lifecycle of the Activity in which the Fragment lives directly affects the lifecycle of the Fragment.

onPause (Activity) → onPause (Fragment)

Another activity is in the foreground and has focus, containing activity is visible
כאשר נפתחת Activity אחרת מעל (לדוגמה – דיאלוג או ניווט), ה- Fragment מאבד מיקוד.

onResume(Activity) → onResume (Fragment)

Fragment is visible in the running activity

כאשר ה- Activity חוזרת להיות פעילה, ה- Fragment מוצג שוב ופעיל למשתמש

onStart (Activity) → onStart (Fragment)

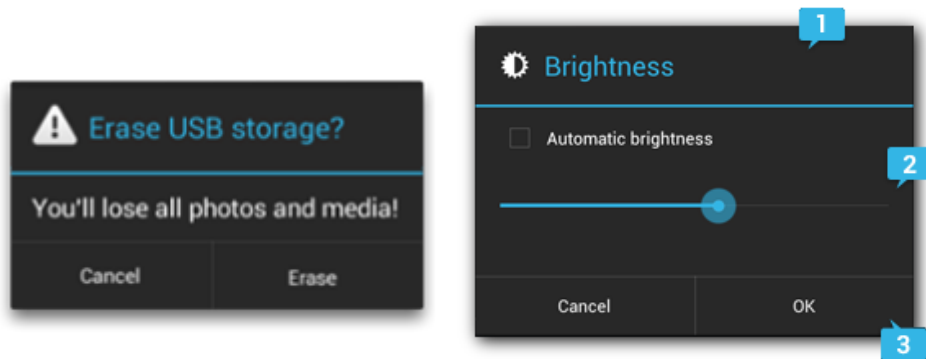
השלב שבו ה- Activity נראית – אך עדיין לא אינטראקטיבית במלואה

onDestroy (Activity) → onDestroy (Fragment)

כשה- Activity נהרסת, גם ה- Fragment נהרס, כולל שחרור משאבים.

Dialogs - AlertDialog

- A dialog is a small window that prompts the user to make a decision or enter additional information.
- A dialog does not fill the screen and is normally used for modal events that require users to take an action before they can proceed.
- A dialog that can show a title, up to three buttons, a list of selectable items, or a custom layout.
- The [AlertDialog](#) class allows you to build a variety of dialog designs and is often the only dialog class you'll need.



- Dialog הוא חלון מוקפץ קטן (לא במסך מלא) שמבקש מהמשתמש לבצע פעולה או להזין מידע.
- משמש בעיקר לאירועים שדורשים תגובה מיידייה מהמשתמש לפני המשך עבודה.
- דיאלוג יכול לכלול:
 - כותרת Title
 - אזור תוכן, Message, List, או Layout מותאם
 - עד 3 כפתורי פעולה (למשל: אישור, ביטול, עזרה)

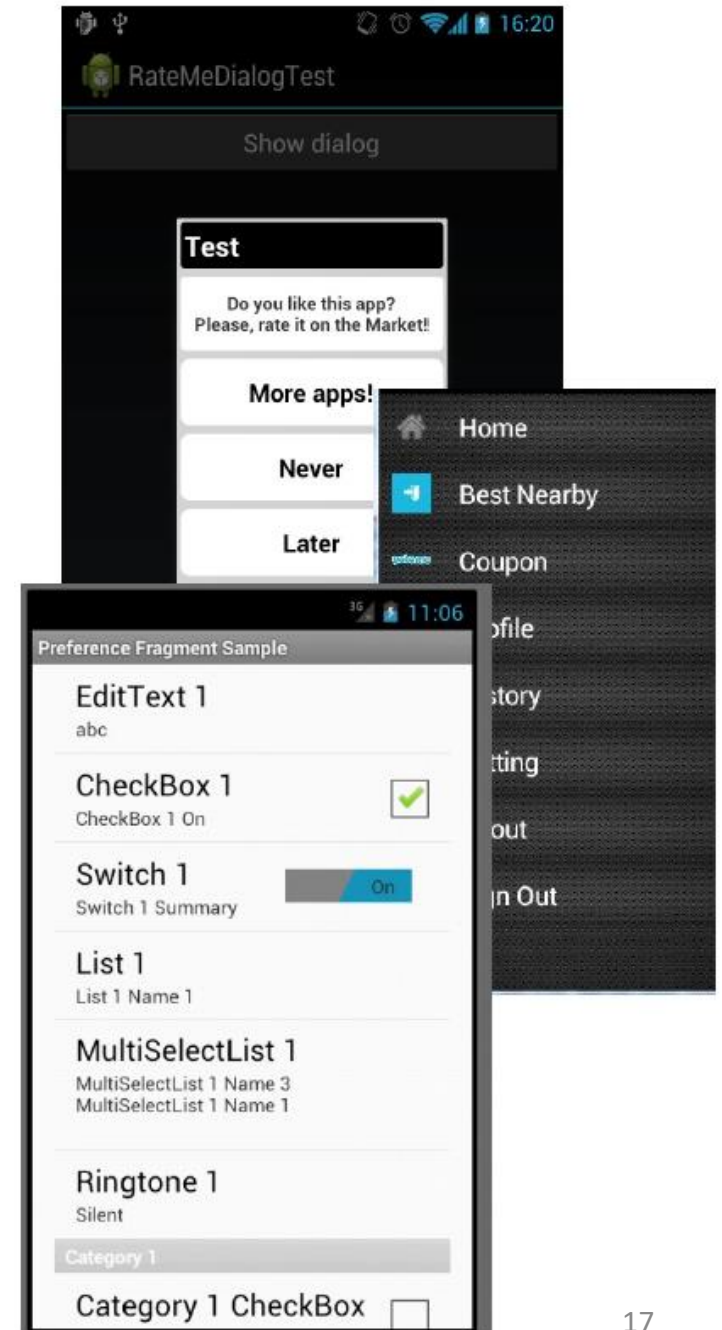
- 1. Title:** This is optional and should be used only when the content area is occupied by a detailed message, a list, or custom layout.
- 2. Content area:** This can display a message, a list, or other custom layout.
- 3. Action buttons:** There should be no more than three action buttons in a dialog.

שימוש נפוץ: בקשות אישור, אזהרות,
בחירת פריטים או קבלת החלטות
ממשתמש.

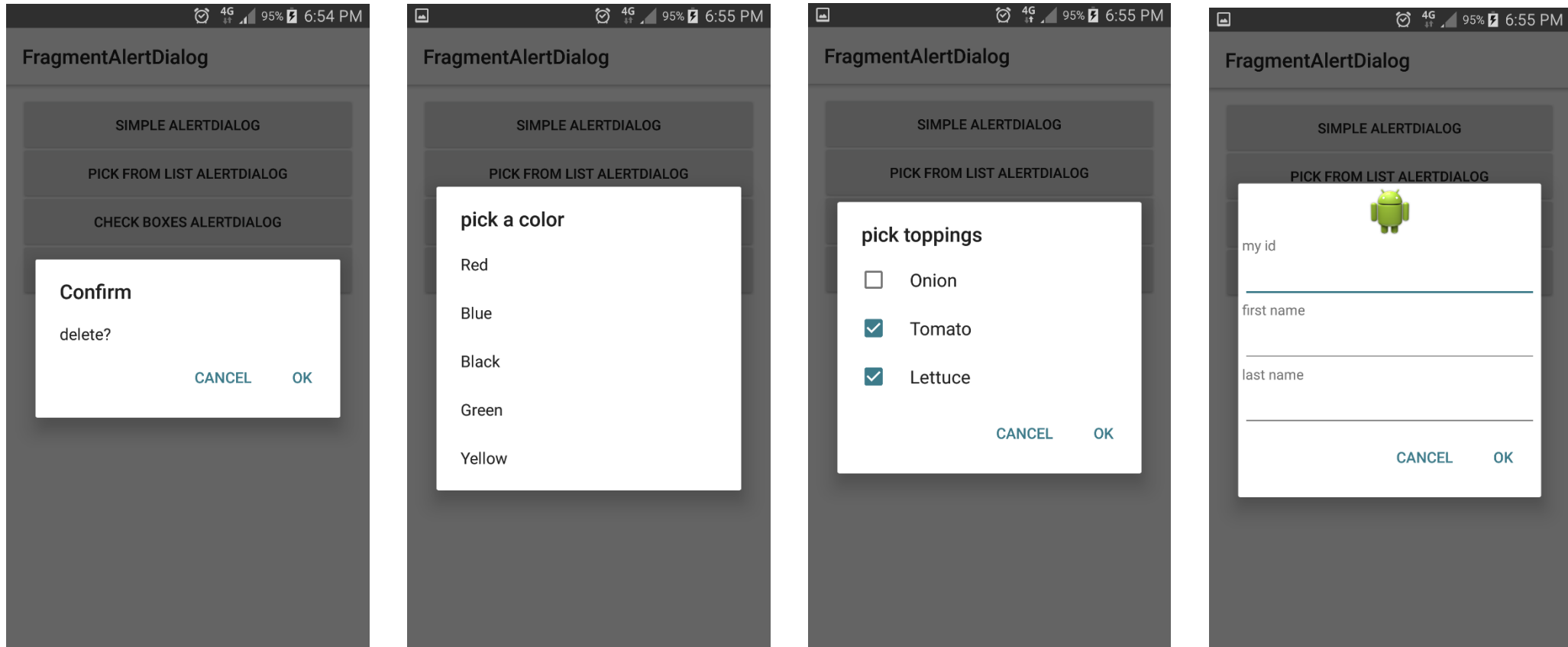
Fragment subclasses

- **DialogFragment**
- a fragment meant to be shown as a dialog box that pops up on top of the current activity.

• DialogFragment הוא סוג מיוחד של Fragment שנועד להציג חלון דיאלוג
• הדיאלוג מוצג מעל ה-Activity הנוכחית, אך נשלט דרך מנגנון ה-Fragment



Dialogs - AlertDialog



Let's look at the code of the xml of the Activity.

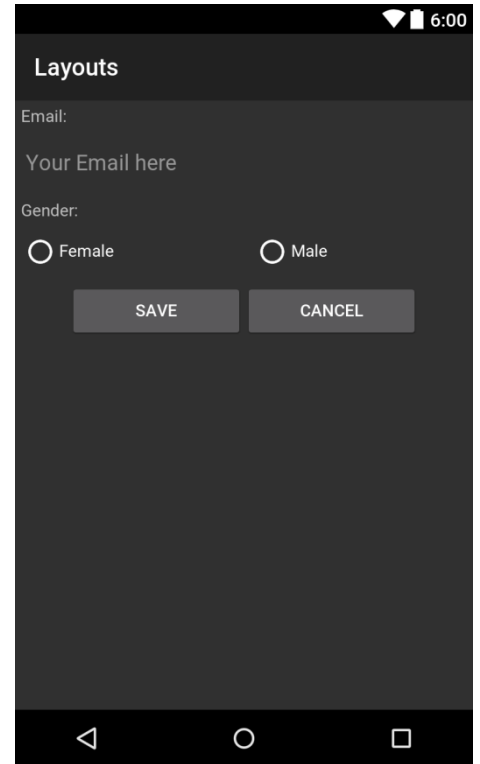
Import project called **FragmentAlertDialog.zip** (download from our course website) into Android Studio

SharedPreferences – store simple data in device

SharedPreferences הוא מנגנון אחסון קל משקל לשמירת מידע בסיסי במכשיר, כמו:

- מצב האפליקציה למשל: אם המשתמש מחובר הגדרות משתמש כגון שפה, מצב כהה
- נתונים מותאמים אישית כמו שם משתמש או מגדר
- הנתונים נשמרים כזוגות של: מפתח ← ערך

- Many applications need a lightweight data storage mechanism for storing:
- Application state
- Configuration options
- User's customizations data: Account name.
- This mechanism on Android platform is called “**Shared Preferences**” and provides a simple preferences system for storing primitive application data at the Activity level.



Let's look at the code of the xml of the following Activity.
Import project called **layouts.zip** (download from our course website) into Android Studio

מתי משתמשים?
• כשצריך לשמור מידע זמני או קבוע פשוט ברמת ה־ Activity או האפליקציה.
• לא מיועד לאובייקטים מורכבים או כמויות גדולות של מידע.

Shared Preferences - Example 1



```
SharedPreferencesReadWriteActivity.java
public class SharedPreferencesReadWriteActivity extends Activity {
    private static String HIGH_SCORE = "high_score";
    private SharedPreferences prefs = null;
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        prefs = getPreferences(MODE_PRIVATE);

        setContentView(R.layout.main);

        // High Score
        final TextView highScore = (TextView) findViewById(R.id.high_score_text);
        highScore.setText(String.valueOf(prefs.getInt(HIGH_SCORE, 0)));

        //Game Score
        final TextView gameScore = (TextView) findViewById(R.id.game_score_text);
        gameScore.setText(String.valueOf("0"));

        // Play Button
        final Button playButton = (Button) findViewById(R.id.play_button);
        playButton.setOnClickListener(new OnClickListener() {

            @Override
            public void onClick(View v) {

                Random r = new Random();
                int val = r.nextInt(1000);
                gameScore.setText(String.valueOf(val));

                // Get Stored High Score
                if (val > prefs.getInt(HIGH_SCORE, 0)) {

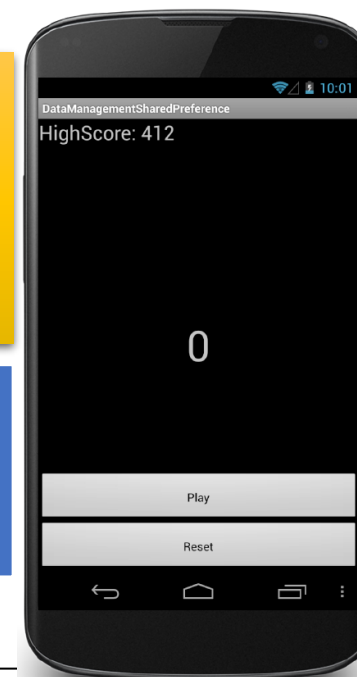
                    // Get and edit high score
                    SharedPreferences.Editor editor = prefs.edit();
                    editor.putInt(HIGH_SCORE, val);
                    editor.commit();

                    highScore.setText(String.valueOf(val));

                }
            }
        });
    }
}
```

•מאחסן את הנתונים כך שרק האפליקציה שלך תוכל לגשת אליהם.
•מונע מגישה של אפליקציות אחרות לקרוא או לשנות את הנתונים האלה.
•זהו מצב ברירת המחדל) ולמעשה גם המומלץ (ברוב המקרים).

בעבר היו מצבים כמו
MODE_WORLD_READABLE
ו־MODE_WORLD_WRITEABLE, שאפשרו
שיתוף בין אפליקציות, אבל הם הוסרו מגרסאות
מודרניות של Android בגלל סכנות אבטחה .



```
// Reset Button
final Button resetButton = (Button) findViewById(R.id.reset_button);
resetButton.setOnClickListener(new OnClickListener() {

    @Override
    public void onClick(View v) {

        // Set high score to 0
        SharedPreferences.Editor editor = prefs.edit();
        editor.putInt(HIGH_SCORE, 0);
        editor.commit();

        highScore.setText(String.valueOf("0"));
        gameScore.setText(String.valueOf("0"));

    }
});
```

Lets look at the code of the xml of the following Activity. Import project called **DataManagementSharedPreferences1.zip** (download from our course website) into Android Studio

Shared Preferences - Example 2



```
public class Calc extends Activity {  
    public static final String PREFS_NAME = "MyPrefsFile";
```

```
    @Override  
    protected void onCreate(Bundle state){  
        super.onCreate(state);  
        . . .
```

```
        // Restore preferences  
        SharedPreferences settings = getSharedPreferences(PREFS_NAME, 0);  
        boolean silent = settings.getBoolean("silentMode", false);
```

Read Shared
Preferences

or **MODE_PRIVATE**

Other get
methods are:
getFloat(...),
getInt(...),
getLong(...),
getString(...)

default value

```
    @Override  
    protected void onStop(){  
        super.onStop();
```

Edit/Add Shared
Preferences

```
        SharedPreferences settings = getSharedPreferences(PREFS_NAME, 0);  
        SharedPreferences.Editor editor = settings.edit();  
        editor.putBoolean("silentMode", mSilentMode);  
        // Commit the edits!  
        editor.commit();  
    }
```

Other put
methods are:
putFloat(...),
putInt(...),
putLong(...),
putString(...)

Commit all
changes from this
editing session.